



Stock Price Prediction

Prepared By: Palwasha Mushtaq

Internship Task : 2

Month : 1

Tools Used : Python, Jupyter Notebook,

Internship Domain : Data Science

Submitted To : Arch Technologies





Agenda Slide

- **Data Introduction**
 - **Data Source and Collection**
 - **Data Overview**
 - **Data Cleaning & Preprocessing**
 - **Libraries Used**
 - **Exploratory Data Analysis (EDA)**
 - **Feature Engineering**
 - **Model Building**
 - **Model Evaluation**
 - **Conclusion**
 - **Recommendation**
- 



Data Introduction



Collect Data

Clean & Process

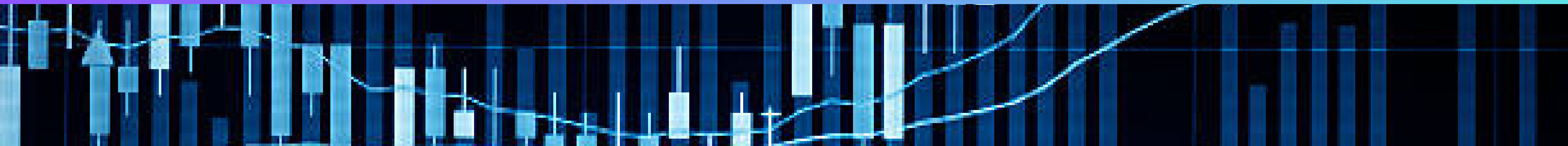
Explore & Visualize

Report Insights

"Stock market prediction is an important area in financial analysis that helps investors make better decisions by understanding price trends. In this project, historical stock data along with sentiment-related features is used to build a prediction model for stock closing prices."

"The dataset includes key features such as Open, High, Low, Close, and Volume, along with sentiment-based variables. After data preprocessing, a Linear Regression model is applied to predict the closing price based on selected numerical features."

"The main goal of this project is to analyze stock market behavior and develop a simple and effective model that can predict price trends with reasonable accuracy."

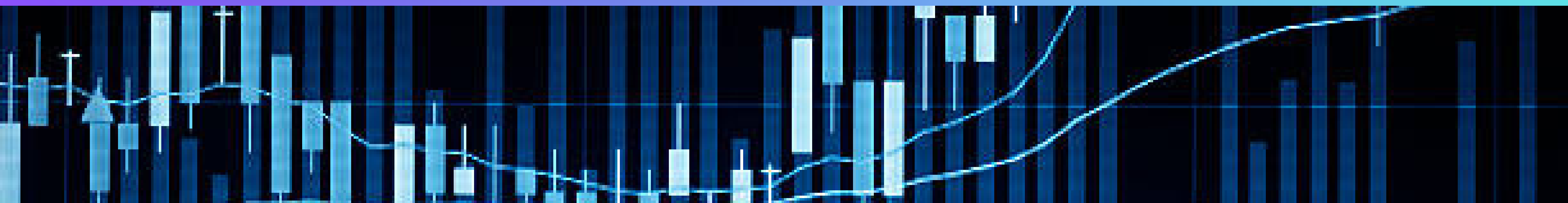




Data Sources & Collection



- **Data Source:** Historical Stock Price Prediction Dataset
- **Collected From:** Online financial data sources (Kaggle)
- **Includes:** Stock price information (Open, High, Low, Close)
- **Trading activity** (Volume)
- **Multiple companies'** data over time
- **Additional Data:** Sentiment-based features for better prediction insights
- **Outcome Variable:** Closing price prediction (Target)

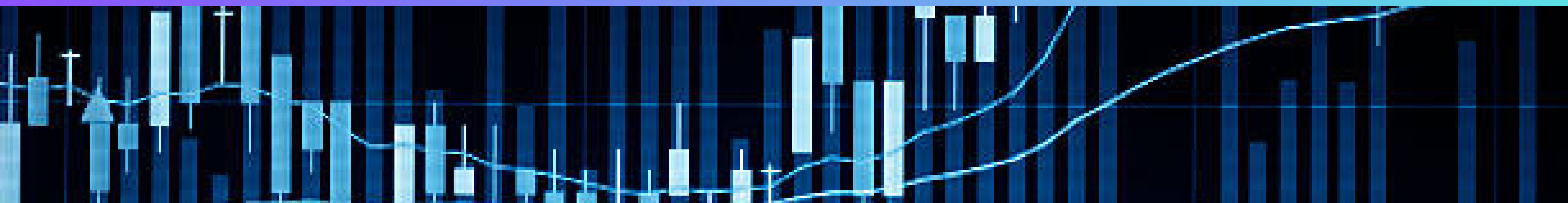


Dataset Overview

Data Profile Summary (Based on .info() Analysis):

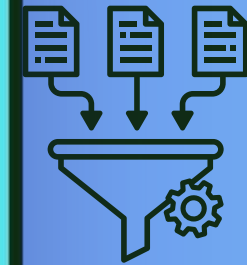
Afer overviwed the data it is understand that:

- Dataset contains 12,069 records and 14 features
- Includes both numerical and categorical variables
- Key features: Open, High, Low, Close, Volume
- Additional features: Company, Date, Sentiment
- Data shows variation in stock prices and trading volume
- Minor missing values found in Cleaned_Text column

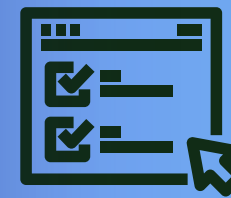


Data Cleaning & Preprocessing

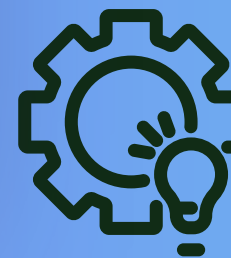
- Dataset checked for missing values
- Only Cleaned_Text column had 154 missing values
- All other columns contained complete data
- Dataset checked for duplicate values
- No duplicate records were found
- Irrelevant columns removed (Company, Cleaned_Text, Sentiment, Sentiment_Score, Comments, Date, Target, Score)
- Focus shifted to key numerical features for modeling
- Data quality ensured before model training



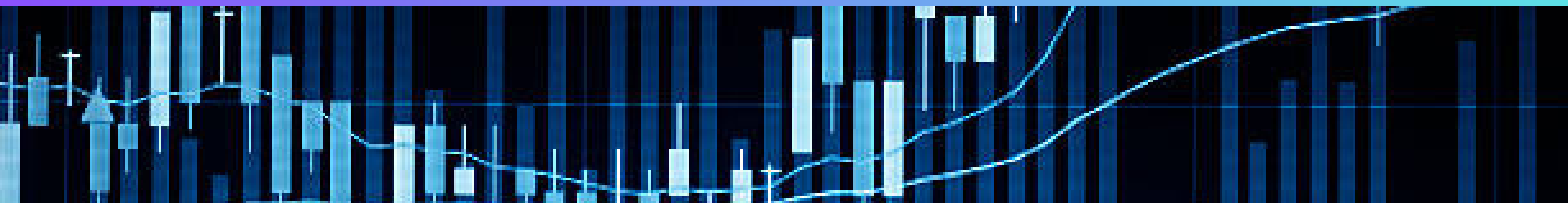
Remove duplicates and errors



Drop Unnecessary Columns



Handle missing values





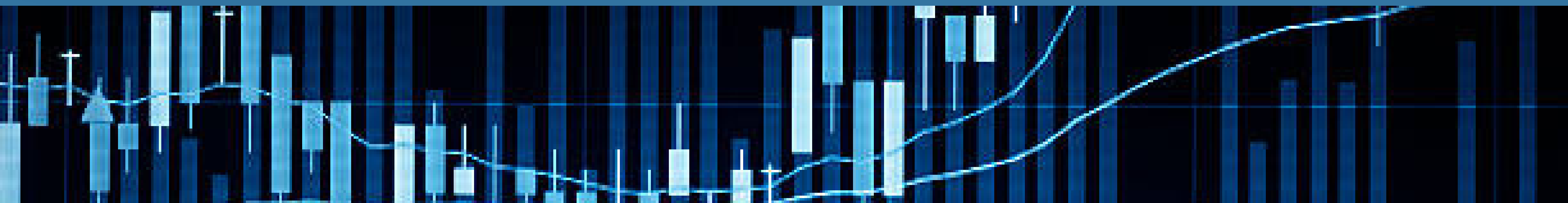
Libraries Used



- Pandas → Data manipulation
- NumPy → Numerical operations
- Matplotlib / Seaborn → Data visualization
- Scikit-learn → Machine learning model

“Scikit-learn was used for model building, training, performance evaluation.”

```
[ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
```



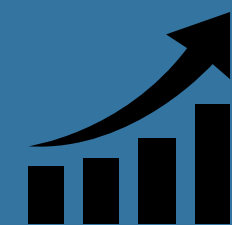


Exploratory Data Analysis (EDA)

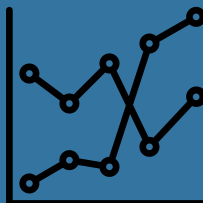


Key Steps:

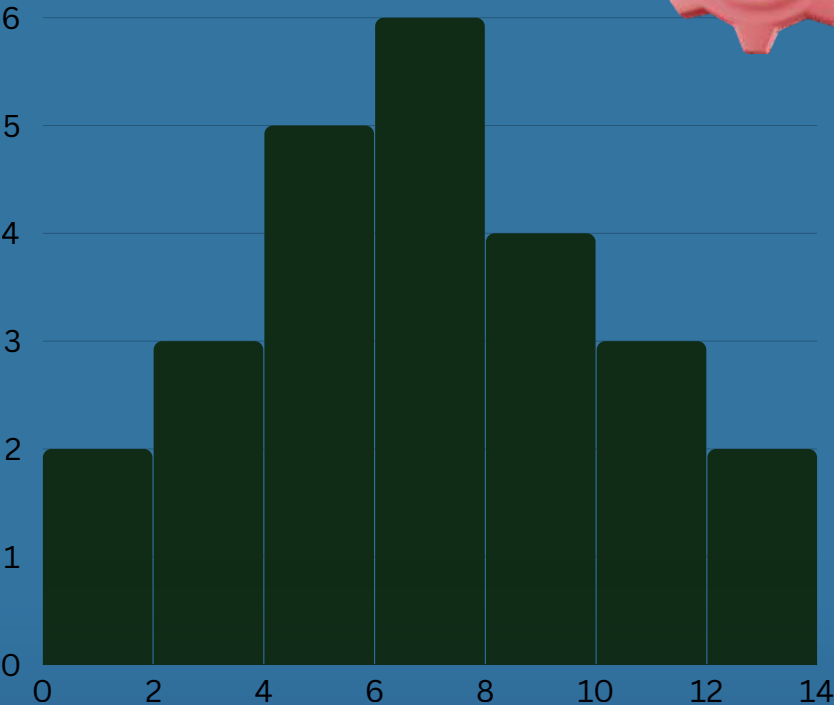
Summarize data



Visualize data



Identify patterns
& correlations



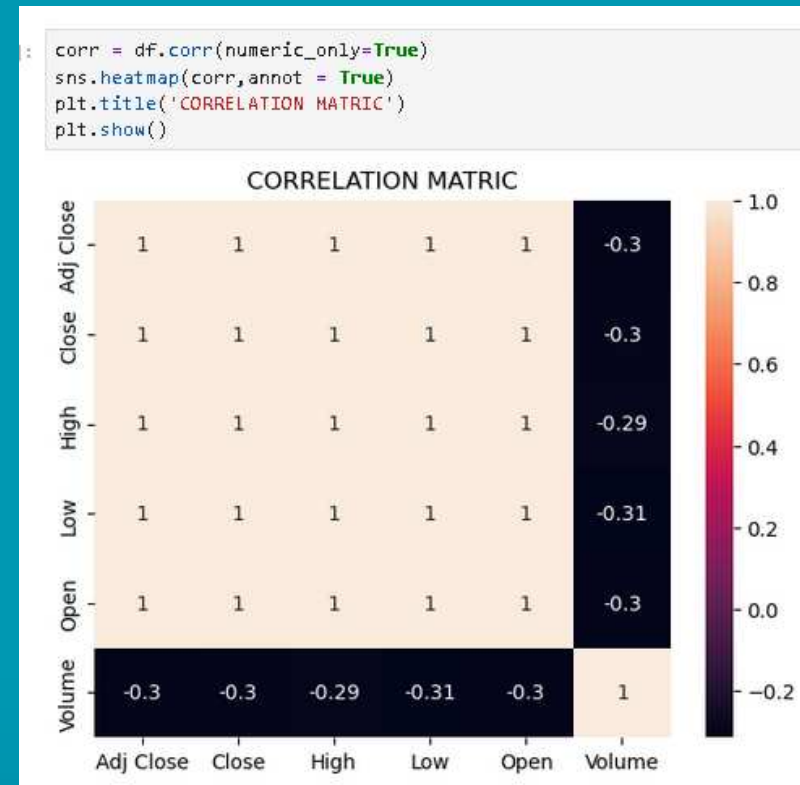
Different visualizations were used to understand the data:

Correlalation Matrix	Closing Price Trend
Trending Volume	Moving Average Price



Data Visualization

Correlation Matrix



"The correlation matrix shows a strong positive relationship between stock price features such as Open, High, Low, Close, and Adjusted Close, indicating that these variables move closely together. In contrast, Volume shows a weak negative correlation with price features."

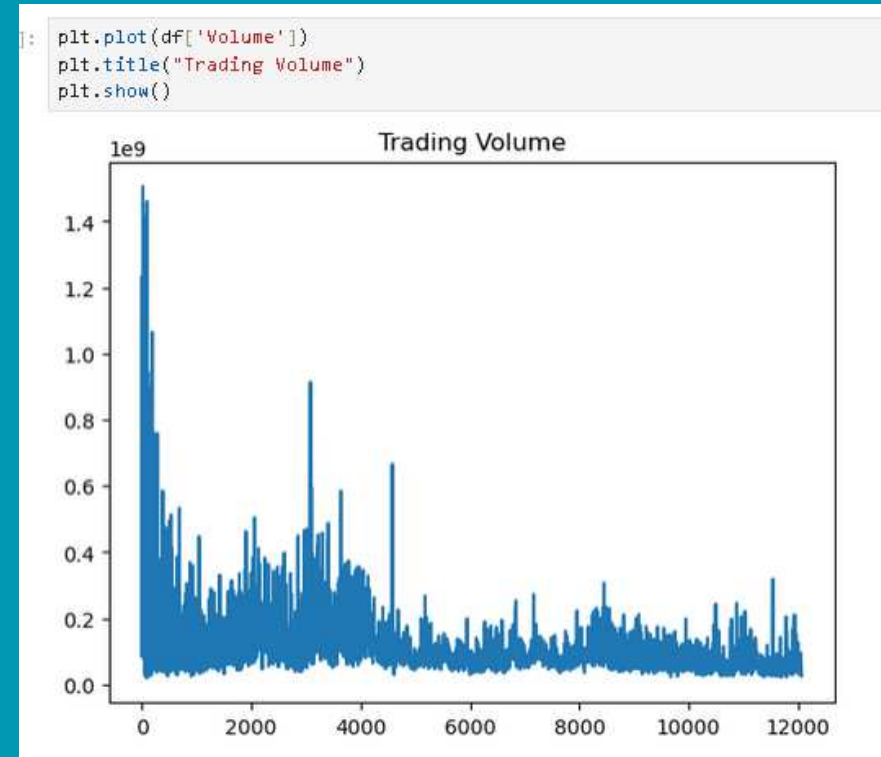
Closing Trend



The graph shows the trend of stock closing prices over time. It can be observed that the prices fluctuate significantly, indicating the volatile nature of the stock market. However, an overall upward trend can be seen in certain periods."

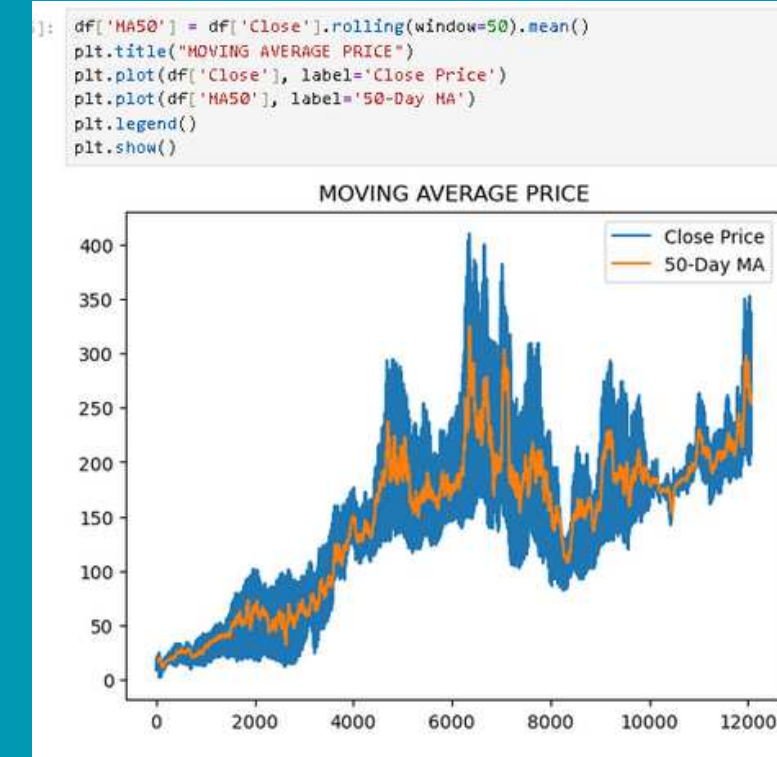
Data Visualization

Trending Volume



"Trading Volume over time. X-axis shows sequential trading sessions (0-12000), Y-axis shows volume in billions. Initial period shows extremely high trading activity with volume >1.4B, followed by declining trend with intermittent spikes."

Moving Average



"The 50-day moving average smooths out short-term fluctuations in stock prices and highlights the overall trend. The moving average line closely follows the price trend but with less volatility."



Feature Engineering



- "Relevant numerical features including Open, High, Low, and Volume were used as independent variables, while the Close price was selected as the dependent variable for prediction."
- "Training data (80%) was used to train the model on historical stock price patterns and Testing data (20%) was used to check how well the model performs on new, unseen data."

```
X = df[['Open', 'High', 'Low', 'Volume']]  
y = df['Close']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, shuffle=False)  
print('TRAINING DATA')  
print(X_train)  
print('TEST DATA')  
print(X_test)
```

```
TRAINING DATA  
   Open      High      Low  Volume  
0  9.860000  10.135000  9.851786  658677600  
1  8.816500   8.897000  8.686500   84050000  
2  8.686000   8.950000  8.679500  116210000  
3  8.843000   8.987500  8.728000   93130000  
4  13.321429  13.602857  13.282143  467832400  
...      ...      ...      ...  
9650 189.570007 190.960007 188.649994  54412900  
9651 140.910004 143.320007 139.520004  49653500  
9652 140.910004 143.320007 139.520004  49653500  
9653 140.910004 143.320007 139.520004  49653500  
9654 140.910004 143.320007 139.520004  49653500
```

```
[9655 rows x 4 columns]  
TEST DATA  
   Open      High      Low  Volume  
9655 140.910004 143.320007 139.520004  49653500  
9656 142.660004 145.229996 142.539993  49636700  
9657 142.660004 145.229996 142.539993  49636700  
9658 232.000000 237.389999 226.539993 142532800  
9659 232.000000 237.389999 226.539993 142532800  
...      ...      ...      ...  
12064 206.979996 207.639999 205.050003  28061600  
12065 206.979996 207.639999 205.050003  28061600  
12066 205.830002 208.199997 204.589996  24892400  
12067 234.809998 237.809998 233.970001  28481400  
12068 234.809998 237.809998 233.970001  28481400
```


Model Building - Linear Regression

```
model = LinearRegression()  
model.fit(X_train,y_train)
```

LinearRegression

Parameters

- “A Linear Regression model was initialized and trained using the training data (X_train and y_train) to learn the relationship between input features and the target variable.”.

```
y_pred = model.predict(X_test)  
print(y_pred)  
  
[141.86705571 144.82358563 144.82358563 ... 206.83054291 236.69689479  
236.69689479]
```

- “The model outputs a continuous range of predicted stock prices, indicating the estimated future values based on learned relationships from historical data.”



Model Evaluation

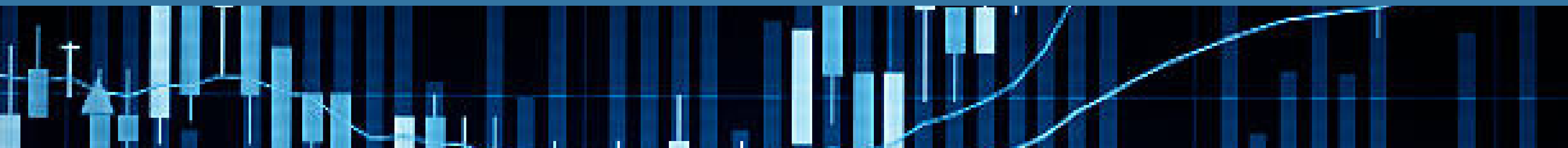


"The model was evaluated using MAE and MSE metrics. The Mean Absolute Error was 1.1766, which shows the average absolute deviation between predicted and actual stock prices. The Mean Squared Error was 2.8544, indicating that the model maintains a low level of overall prediction error and performs reasonably well on the dataset."

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
print("MEAN_ABSOLUTE_ERROR:", mae)
print("MEAN_SQUARED_ERROR:", mse)
```

```
MEAN_ABSOLUTE_ERROR: 1.176609776004132
MEAN_SQUARED_ERROR: 2.8544591874014316
```

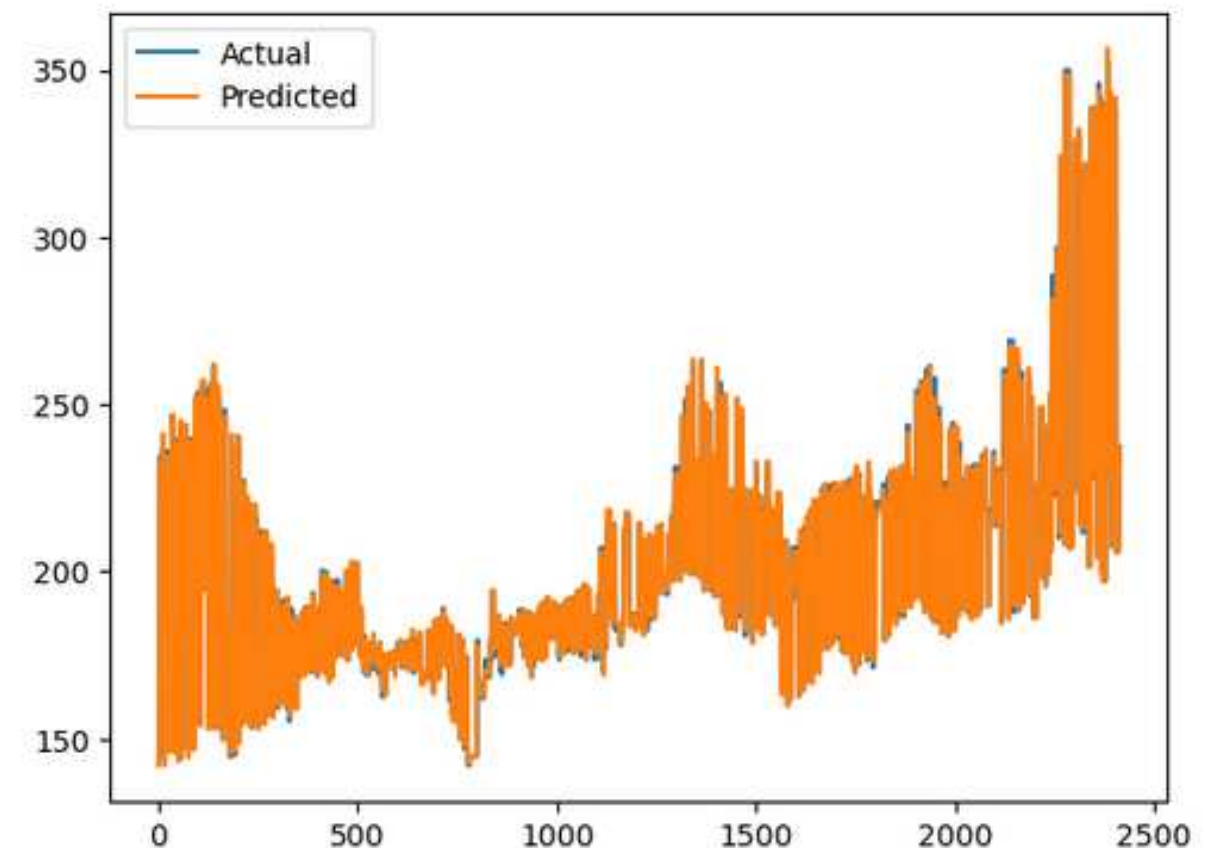


Model Evaluation

"The model shows high accuracy with predicted values (orange) overlapping actual values (blue)."

"Trend and volatility are well captured with minimal deviation, confirming good model performance on test data."

```
plt.plot(y_test.values, label="Actual")  
plt.plot(y_pred, label="Predicted")  
plt.legend()  
plt.show()
```



Conclusion

- This project follows a complete end-to-end data science pipeline including preprocessing, EDA, feature engineering, visualization, and model building.
- The stock price prediction model was developed using a dataset of 12,069 observations and 14 features, including key stock market indicators such as Open, High, Low, Close, and Volume along with sentiment-based variables.
- During data preprocessing, irrelevant and text-based columns were removed to focus on numerical features, ensuring better model performance and accuracy.
- A Linear Regression model was trained using Open, High, Low, and Volume as input features to predict the Close price. The dataset was split into training and testing sets for proper evaluation.
- Model evaluation showed good performance with a Mean Absolute Error (MAE) of 1.17 and Mean Squared Error (MSE) of 2.85, indicating low prediction error and reliable short-term forecasting ability.
- Visualization analysis further supported the model performance: The Closing Price Trend graph showed overall market movement over time.
- The Trading Volume trend highlighted fluctuations in market activity.
- The Moving Average analysis helped smooth short-term volatility and clearly showed overall trend direction.
- The Correlation analysis confirmed strong relationships between selected features and the target variable (Close price).
- The Actual vs Predicted plot demonstrated that predicted values closely follow actual stock prices with minimal deviation.
- Overall, the model successfully captures market trends and shows strong predictive behavior, making it suitable for short-term stock price estimation with basic linear regression techniques.





Recommendation



- "More advanced machine learning models such as Random Forest, XGBoost, or LSTM (deep learning) can be applied to further improve prediction accuracy."
 - "Incorporating technical indicators (e.g., Moving Averages, RSI, MACD) may enhance model performance."
 - "Sentiment analysis features should be properly integrated, as market sentiment can significantly influence stock prices."
 - "More feature engineering (lag features, rolling averages) can help capture time-series dependencies better."
 - "Hyperparameter tuning and cross-validation should be applied to ensure better generalization of the model."
 - "Future work can focus on time-series forecasting models instead of simple regression for improved long-term predictions."
- 



Thank You

THANK YOU FOR YOUR ATTENTION

